

CH02 Spring IoC 原理和实现

考点复习与总结

基于课程课件整理 | 共 42 页内容全覆盖

本文档整理了 CH02 Spring IoC 原理和实现课程课件的所有考点，按专题组织，每条要点标注课件页码引用 [p.X]。中文为主，关键术语附英文原名。

目录

1	程序架构的发展与 IoC 思想的产生	3
1.1	从 HelloWorld 到企业级架构的演进	3
1.2	简单工厂模式——初步解耦尝试	3
1.3	IoC 思想的萌芽	3
1.4	程序架构演进路线总结	3
2	IoC 基本概念与核心原理	4
2.1	IoC 的定义与特点	4
2.2	IoC 核心原理	4
2.3	IoC 原理范例——开发打印机	4
3	Spring IoC 容器详解	5
3.1	Bean 的概念	5
3.2	核心接口对比：BeanFactory 与 ApplicationContext	5
3.3	常见 ApplicationContext 实现	5
3.4	Spring IoC 容器使用步骤（XML 配置方式）	5
4	Bean 的定义和配置方式	5
4.1	XML 配置（Spring 1.x 起）	5
4.2	注解配置（Spring 2.5+ 起）	6
4.3	Java Config（Spring 3.0+ 起）	6
4.4	三种配置方式对比	6
5	依赖注入（Dependency Injection, DI）	6
5.1	构造器注入（Constructor Injection）	6
5.2	Setter 注入（Setter Injection）	7
5.3	字段注入（Field Injection）	7
5.4	三种 DI 方式对比总结	7
5.5	处理多个同类型 Bean 的注入	8
6	Bean 的作用域（Bean Scopes）	8
6.1	Singleton 与 Prototype 关键区别	8
7	重点注解速查表	9

8 核心考点速记	9
8.1 IoC 与 DI 的关系	9
8.2 Spring IoC 容器的核心机制	9
8.3 常见易错点	10
9 本章小结	10

1 程序架构的发展与 IoC 思想的产生

1.1 从 HelloWorld 到企业级架构的演进

- **HelloWorld 经典程序（阶段 1）**：最简形式的 Java 程序，所有逻辑在主函数完成，输出内容硬编码在代码中。[p.7]
- **命令行参数版本（阶段 2）**：当需要修改输出内容时，最简方式是在代码中修改字符串，但需重新编译和测试。更好方案：让程序从外部接受字符串，运行时决定显示内容。[p.8]
- **面向对象 OOP 版本（阶段 3）**：程序规模扩大后引入面向对象设计，包含三个功能模块：[p.9]
 - 模块一：从持久化数据（文件、数据库等）中读取字符串；
 - 模块二：对读取的数据进行处理（编码、格式等）；
 - 模块三：将数据展现在用户界面（命令行、GUI、HTML 等）。
- 对应三个类分别承担不同职责：[p.9–11]
 - `FileHelloStr`：从持久化数据（文件）中读取信息；
 - `HelloWorld`：业务逻辑，如添加 `Software` 后缀；
 - `HelloWorldClient`：将信息输出到命令行。
- **高耦合问题分析**：[p.13]
 - `HelloWorld` 类依赖于 `FileHelloStr` 类；
 - `HelloWorldClient` 类依赖于 `HelloWorld` 类；
 - 系统各部分之间相互依赖，导致可维护性和扩展性差，不适合大型、企业级应用开发。

1.2 简单工厂模式——初步解耦尝试

- **简单工厂模式（Simple Factory Pattern）**：将依赖对象的创建集中到工厂中，业务类不再直接依赖具体实现类，只需依赖接口和工厂。[p.14–15]
- **优点**：[p.15]
 - 将依赖对象的创建集中到工厂中，降低直接耦合；
 - `OrderService` 不再直接依赖具体实现类，只需依赖 `PaymentService` 接口和工厂。
- **局限**：[p.15]
 - 工厂本身仍需要修改（新增支付方式时需修改工厂代码）；
 - 工厂成为新的依赖点，如果系统有几十个这样的工厂，管理将变得混乱；
 - 工厂通常是静态方法，难以替换为 `Mock` 对象进行测试。

1.3 IoC 思想的萌芽

- **解耦思路**：将“创建和管理对象”的职责完全从业务代码中抽离出来，交给一个通用的基础设施。[p.16]
- **IoC（控制反转）思想的雏形**：[p.16]
 - 将对象的控制权交给容器（Container）；
 - 使用高效、通用的外部工厂（框架、容器）统一管理对象；
 - 借助依赖注入（Dependency Injection, DI）实现控制反转；
 - 通过配置文件替代硬编码；
 - 通过反射等技术自动创建对象。

1.4 程序架构演进路线总结

阶段	架构方案	解决的核心问题
阶段 1	简单 Main 函数	基本功能实现 [p.7]
阶段 2	参数化	运行时配置灵活性 [p.8]
阶段 3	面向对象 OOP	模块化、职责分离 [p.9–11]
阶段 4	简单工厂模式	对象创建的集中管理 [p.14–15]
阶段 5	IoC/DI	彻底解耦、依赖外部管理 [p.16]

2 IoC 基本概念与核心原理

2.1 IoC 的定义与特点

- **控制反转（Inversion of Control, IoC）**：是面向对象编程中的一种设计原则，可以用于降低计算机代码之间的耦合度。[p.18]
- IoC 常用的实现方式是**依赖注入（Dependency Injection, DI）**。[p.18]
- 通过控制反转，对象在被创建时，由系统的外部实体将其所依赖的对象的引用传递（注入）进来，而不是系统内部进行对象创建。[p.18]
- 目前最常见的具体实现是 **Spring Framework**。[p.18]

2.2 IoC 核心原理

- **核心思想**：解耦合——将组件的**构建**和组件的**使用**分开，实现每个组件时只考虑组件内部的事情。[p.19]
- **目标**：明确和定义组件间的接口，实现**高内聚、低耦合**。[p.19]
- **实现基础**：面向接口编程 + 工厂模式 + 依赖注入。[p.19]
- 四个关键环节：[p.19]
 1. 接口的定义
 2. 组件的生产
 3. 组件的使用
 4. 运行时注入

2.3 IoC 原理范例——开发打印机

- 以开发一台打印机为例说明 IoC 原理，打印机功能实现依赖于墨盒和纸张。[p.20–24]
1. **定义组件接口** [p.21]
 - 墨盒接口：Ink
 - 纸张接口：Paper
 2. **使用接口开发打印机**：打印机类依赖 Ink 和 Paper 接口，而非具体实现。[p.22]
 3. **组装打印机——添加实现类**：编写 Ink 和 Paper 的具体实现类（如彩色墨盒、A4 纸张等）。[p.23]
 4. **使用 Spring IoC 容器装配对象**：通过配置文件将实现类注入到打印机中，由容器负责组装。[p.24]

该范例体现了 IoC 的精髓：打印机开发者只需关注接口，具体的墨盒和纸张实现由外部容器在运行时注入。

3 Spring IoC 容器详解

3.1 Bean 的概念

- **Bean:** 由 Spring IoC 容器负责实例化、组装和管理的对象称为 Bean。[p.25]
- Bean 就是普通的 Java 对象（POJO），但被纳入了容器的管理范围。[p.25]

3.2 核心接口对比：BeanFactory 与 ApplicationContext

Spring IoC 容器有两个核心接口：[p.25]

特性	BeanFactory	ApplicationContext
层次定位	Spring IoC 容器的顶层接口，提供最基础的 IoC 功能	BeanFactory 的子接口，企业级应用的首选
加载策略	延迟加载（Lazy Loading），仅在调用 <code>getBean()</code> 时才实例化 Bean	立即加载（Eager Loading），容器启动时预加载所有单例 Bean
适用场景	资源受限的环境（如移动设备）	企业级应用开发
企业级功能	基础功能	国际化（ <code>MessageSource</code> ）、事件发布（ <code>ApplicationEvent</code> ）、AOP 集成、环境抽象等

3.3 常见 ApplicationContext 实现

- `ClassPathXmlApplicationContext`: 从类路径加载 XML 配置文件。[p.25]
- `FileSystemXmlApplicationContext`: 从文件系统加载 XML 配置文件。[p.25]
- `AnnotationConfigApplicationContext`: 基于 Java 配置类和注解。[p.25]

3.4 Spring IoC 容器使用步骤（XML 配置方式）

1. 导入 Spring IoC 相关的依赖包（如 `spring-context` 等 JAR 包）。[p.27]
2. 创建配置文件 `applicationContext.xml`。[p.28]
3. 添加 Bean 的 XML 配置：在 XML 中声明 Bean 及依赖关系。[p.29]
4. 初始化容器，并使用容器装配对象：[p.30]
 - 创建 `ApplicationContext` 实例并加载配置；
 - 通过 `getBean()` 方法获取已装配好的对象。

4 Bean 的定义和配置方式

Spring IoC 容器提供三种定义和配置 Bean 的方式。[p.32]

4.1 XML 配置（Spring 1.x 起）

- 优点：[p.32]
 - 完全解耦：所有 Bean 关系在 XML 中定义；
 - 修改无需重新编译代码。

- 缺点: [p.32]
 - 配置冗长;
 - 维护成本高;
 - 难以管理大量 Bean。

4.2 注解配置 (Spring 2.5+ 起)

- 在类上使用 `@Component`、`@Service`、`@Repository`、`@Controller` 等注解声明 Bean。[p.32]
- 在字段/方法上使用 `@Autowired` 或 `@Resource` 进行依赖注入。[p.32]
- 优点: [p.32]
 - 开发效率高;
 - 配置与代码结合紧密, 直观。
- 缺点: [p.32]
 - Bean 的依赖关系分散在代码中;
 - 修改可能涉及重新编译。

4.3 Java Config (Spring 3.0+ 起)

- 使用 `@Configuration` 标记配置类。[p.32]
- 使用 `@Bean` 标记方法返回 Bean 实例。[p.32]
- 优点: [p.32]
 - 类型安全 (Type Safe);
 - 支持重构;
 - 可灵活编程 (如条件判断创建 Bean)。
- 未来 Spring Boot 基于该方式进一步简化了配置。[p.32–33]

4.4 三种配置方式对比

对比维度	XML 配置	注解配置	Java Config
Spring 版本	1.x 起	2.5+ 起	3.0+ 起
解耦程度	完全解耦	耦合在代码中	耦合在配置类中
类型安全	无 (字符串)	有	有
修改需编译	否	是	是
配置冗长度	高	低	中等
开发效率	低	高	中等
可重构性	差	一般	好
条件编程	不支持	不支持	支持

5 依赖注入 (Dependency Injection, DI)

Spring IoC 支持三种依赖注入方式。[p.34]

5.1 构造器注入 (Constructor Injection)

- 通过构造函数的参数注入依赖。[p.35]

- 优点: [p.35]
 - 强制性: 对象必须提供依赖才能被创建, 避免了空指针异常;
 - 不可变性: 依赖一旦注入, 在对象生命周期内不会改变, 适合线程安全的场景;
 - 易于测试: 在单元测试中可直接 `new OrderService(mockUserDao, mockPaymentService)`, 无需依赖 Spring 容器;
 - 明确依赖关系: 通过构造器参数清晰列出所有依赖。
- 缺点: [p.35]
 - 当依赖过多时, 构造器参数列表会变得很长 (这通常是代码需要重构的”坏味道”, 提示该类可能承担了过多职责)。

5.2 Setter 注入 (Setter Injection)

- 通过 Setter 方法注入依赖。[p.36]
- 优点: [p.36]
 - 灵活性: 依赖可以在对象创建后重新设置 (如通过配置变更动态切换实现), 适合可选依赖;
 - 避免循环依赖: 在某些循环依赖场景下, Setter 注入可以配合三级缓存解决 (而构造器注入可能直接抛出异常)。
- 缺点: [p.36]
 - 不确定性: 对象创建后依赖可能为 null (如果忘记调用 setter), 无法保证对象完全初始化;
 - 不适合强制依赖: 如果某个依赖是必须的, 使用 Setter 注入就无法强制要求。

5.3 字段注入 (Field Injection)

- 直接在字段上使用 `@Autowired` 等注解注入依赖。[p.37]
- 注意: 在程序员实际使用中很常见, 但官方文档和组织标准中均不推荐。[p.34]
- 优点: [p.37]
 - 代码极其简洁, 开发效率高。
- 缺点: [p.37]
 - 无法声明 `final`: 依赖是可变的, 破坏了不可变性;
 - 隐藏依赖: 通过类的外部接口 (构造器、setter) 看不到依赖, 容易导致运行时意外 (比如依赖未注入);
 - 与容器强耦合: 脱离 Spring 容器 (如单元测试) 时, 无法简单地通过 `new` 来注入依赖, 必须使用反射或启动容器, 增加测试难度;
 - 容易产生循环依赖: 字段注入容易让人忽略依赖关系, 导致不经意间引入循环依赖。

5.4 三种 DI 方式对比总结

对比维度	构造器注入	Setter 注入	字段注入
强制性	强 (无法创建未注入的对象)	弱 (可能忘记调用 setter)	弱 (依赖可能为 null)
不可变性	支持 (可声明 <code>final</code>)	不支持	不支持 <code>final</code>
可测试性	高 (直接 <code>new</code> 即可)	中	低 (需反射或容器)
循环依赖	可能直接抛异常	可配合三级缓存解决	容易不经意间引入
代码简洁度	中	中	高
官方推荐度	推荐	可选依赖时推荐	不推荐

对比维度	构造器注入	Setter 注入	字段注入
适用场景	强制必需的依赖	可选依赖、需动态切换	应避免使用

5.5 处理多个同类型 Bean 的注入

当一个接口有多个实现类时，注入时需要指定具体使用哪个实现。[p.38-40]

1. 使用 @Primary 标记首选实现 [p.38]

- 在优先使用的实现类上添加 @Primary 注解；
- 当有多个候选 Bean 时，@Primary 标注的 Bean 会被优先注入。

2. 使用 @Qualifier 精确指定 Bean 名称 [p.39]

- 在注入点配合 @Autowired 使用 @Qualifier("beanName")；
- 通过 Bean 的名称（默认为类名首字母小写）精确定位要注入的实现类。

3. 注入全部实现（批量处理） [p.40]

- 使用 List<Interface> 或 Map<String, Interface> 接收所有实现；
- 适用于需要对所有实现进行遍历或选择的场景。

6 Bean 的作用域（Bean Scopes）

Spring IoC 容器中，Bean 可设定不同的作用域，控制其生命周期和可见范围。[p.41]

作用域	说明	适用场景
singleton	默认作用域。每个容器仅创建一个共享实例。Spring 的单例是“容器级单例”，每个容器创建一个实例，而非 JVM 级单例。	无状态服务、DAO 层
prototype	每次请求 Bean 时都创建新实例。容器仅负责初始化，不负责销毁；客户代码需自行清理资源。	有状态或可变对象
request	仅在 Web 环境下有效。每个 HTTP 请求创建一个新实例，请求结束时该实例被销毁。	Web 请求处理
session	仅在 Web 环境下有效。每个 HTTP 会话维持一个实例，在会话失效或超时后销毁。	用户会话信息
application	仅在 Web 环境下有效。与 ServletContext 同生命周期，整个 Web 应用中共享一个实例。	应用级共享数据

6.1 Singleton 与 Prototype 关键区别

- singleton（单例模式）：[p.41]
 - 默认作用域，最常用；
 - 容器启动时即创建实例（配合 ApplicationContext 的立即加载）；
 - 整个容器生命周期内只存在一个实例；
 - 线程安全问题需自行处理。
- prototype（原型模式）：[p.41]
 - 每次 getBean() 都创建新实例；

- 容器只负责初始化，不管理完整生命周期；
- 销毁回调方法不会被自动调用；
- 适用于有状态的 Bean。

7 重点注解速查表

注解	作用层级	用途说明
@Component	类	通用的 Spring 组件注解，将该类标记为 Bean。是 @Service、@Repository、@Controller 的父注解。
@Service	类	标记服务层组件，语义上表示业务逻辑层。
@Repository	类	标记数据访问层组件（DAO），Spring 会自动翻译持久层异常。
@Controller	类	标记控制器组件，用于 Spring MVC 中的请求处理。
@Autowired	字段/构造器/方法	自动注入依赖，默认按类型（byType）匹配。可配合 @Qualifier 按名称精确匹配。
@Qualifier	字段/参数	配合 @Autowired 使用，按 Bean 名称精确指定注入的实现类。
@Primary	类	标记首选的 Bean 实现，当多个同类型 Bean 存在时优先注入。
@Resource	字段/方法	Java 标准的注入注解（JSR-250），默认按名称（byName）匹配。
@Configuration	类	标记该类为配置类（Java Config 方式），替代 XML 配置文件。
@Bean	方法	在 @Configuration 类中使用，标记方法返回值作为 Bean 纳入容器管理。
@Value	字段	注入外部属性值（如 properties 文件中的配置值）。
@Scope	类	指定 Bean 的作用域，如 @Scope("prototype")。

8 核心考点速记

8.1 IoC 与 DI 的关系

- IoC（控制反转，Inversion of Control）是一种设计原则/思想，目标是降低耦合度。[p.18]
- DI（依赖注入，Dependency Injection）是 IoC 的一种常见实现方式。[p.18]
- 关系：IoC 是目标，DI 是实现手段；IoC 是原则，DI 是模式。[p.18-19]

8.2 Spring IoC 容器的核心机制

- Bean 的定义与注册：通过 XML、注解或 Java Config 声明 Bean。[p.32]

- **Bean 的实例化**：容器使用反射创建对象实例。[p.16]
- **依赖注入**：容器自动注入 Bean 所需的依赖。[p.34]
- **Bean 的生命周期管理**：容器负责 Bean 的创建、初始化、使用和销毁。[p.25, 41]
- **Bean 的作用域控制**：singleton、prototype 等作用域决定 Bean 的可见范围和生命周期。[p.41]

8.3 常见易错点

- [易错] Spring 的 singleton 是”容器级单例”，不是 JVM 级单例。如果创建多个 ApplicationContext，每个都会有自己的 singleton 实例。[p.41]
- [易错] prototype Bean 的销毁生命周期回调不会被自动调用，需要客户代码自行管理资源释放。[p.41]
- [易错] 字段注入虽然在代码中很常见，但**官方不推荐**。推荐使用构造器注入用于强制依赖，Setter 注入用于可选依赖。[p.34, 37]
- [易错] BeanFactory 是延迟加载（调用 `getBean()` 时才创建），ApplicationContext 是立即加载（启动时预加载所有单例 Bean）。[p.25]
- [易错] `@Autowired` 默认按类型（byType）注入，当同类型有多个 Bean 时需使用 `@Qualifier` 或 `@Primary` 解决歧义。[p.38–39]

9 本章小结

根据课件本章小结，学习本章后应掌握以下核心内容：[p.42]

1. 为什么需要 IoC？

- 了解程序架构随着业务和代码规模的变化而变化，掌握从 HelloWorld 到 IoC 的演进过程中产生的各种程序架构。[p.7–16, 42]

2. IoC 和 Spring IoC 的核心原理

- 掌握控制反转 IoC 的核心原理（解耦合、将对象创建与使用分离）。[p.19, 42]
- 掌握 Spring IoC 的 Bean 工厂的核心机制（BeanFactory 与 ApplicationContext 的区别）。[p.25, 42]

3. Spring IoC 的相关概念及使用

- Bean 的定义和配置方式（XML 配置、注解配置、Java Config 三种方式）。[p.32, 42]
- 依赖注入 DI 的三种方式（构造器注入、Setter 注入、字段注入）。[p.34–37, 42]
- Bean 的作用域（singleton、prototype、request、session、application）。[p.41, 42]

本文档覆盖原课件全部 42 页内容，按专题组织，便于考前复习与快速查阅。